Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Персистентность и изоляция: добавляем БД и настраиваем сети

Обучающийся: Рекун Ирина Данииловна

Факультет: Факультет прикладной информатики

Группа: K3320

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Рекун И. Д.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ЗАПУСК POSTGRESQL И ПРИЛОЖЕНИЯ 4

1.1 Создание тома и запуск базы данных 4

1.2 Подключение FastAPI к PostgreSQL 5

2 ПРОВЕРКА СОХРАНЕНИЯ ДАННЫХ 6

3 ПОДКЛЮЧЕНИЕ КОНФИГУРАЦИИ 7

4 ПОЛЬЗОВАТЕЛЬСКАЯ СЕТЬ И ИЗОЛЯЦИЯ 9

4.1 Создание сети и подключение контейнеров 9

4.2 Проверка связи между контейнерами 10

5 ИТОГОВАЯ ПРОВЕРКА 12

ЗАКЛЮЧЕНИЕ 13

ВВЕДЕНИЕ

Цель работы — подключаем к приложению PostgreSQL, учимся хранить данные вне контейнера и разбираемся с сетями Docker.

Работа выполнена в Ubuntu WSL с использованием Docker Desktop. Продолжен проект второй лабораторной работы: приложение FastAPI расположено в каталоге solutions/assist-py/, изменения сохранены в ветке lab-3 личного форка.

Последние две цифры номера ИСУ — 28. Порт PostgreSQL на хосте рассчитан как $5430 + 28 = 5458$, порт приложения — как $8000 + 28 = 8028$. Персональные параметры приведены в таблице 1.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	467228
Порт PostgreSQL на хосте	5458
Порт приложения на хосте	8028
Персональный префикс	rekun
Учебный секретный ключ	secret28rekun
Имя базы данных	rekun28

Для базы данных используется контейнер rekun-postgres, для приложения — rekun-assistant-lab3. Созданы именованный том rekun-pgdata и пользовательская сеть rekun-backend.

Репозиторий и версия приложения для третьей работы: https://github.com/grettl/course-containerization-orchestration-application/tree/lab-3.

1 ЗАПУСК POSTGRESQL И ПРИЛОЖЕНИЯ

1.1 Создание тома и запуск базы данных

Для хранения файлов PostgreSQL создан именованный том rekun-pgdata. Его наличие подтверждено командой docker volume ls (рисунок 1). Том существует отдельно от файловой системы контейнера.

```
ntainerization-orchestration-application/solutions/assist-py$ git switch lab-3
Переключились на ветку «lab-3»
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/course-co
ntainerization-orchestration-application/solutions/assist-py$ docker volume create re
kun-pgdata
rekun-pgdata
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/course-co
ntainerization-orchestration-application/solutions/assist-py$ docker volume ls
DRIVER    VOLUME NAME
local     10bd23788aee68eb58a7576c4eb8756137cdadcb6df908f19fe8ef62e8fc34c4
local     840485da58a6932009ae5a0138b7a05883cbb8607f1fb659e02a75535cb13cd8
local     rekun-pgdata
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/course-co
ntainerization-orchestration-application/solutions/assist-py$
```

Рисунок 1 — Создание и проверка тома

Команда docker volume create создаёт том, а docker volume ls показывает список томов. Docker управляет местом их хранения. При удалении контейнера именованный том остаётся, если его не удалить отдельно.

Контейнер rekun-postgres запущен на образе postgres:16. Том подключён к каталогу /var/lib/postgresql/data. Порт хоста 5458 сопоставлен с внутренним портом базы 5432 (рисунок 2).

```
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/course-containerization-orc
hestration-application/solutions/assist-py$ docker ps --filter name=rekun-postgres
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS
                         NAMES
160740366c5f   postgres:16   "docker-entrypoint.s…"   4 seconds ago   Up 4 seconds   0.0.0.0:5458->5432
/tcp, [::]:5458->5432/tcp   rekun-postgres
```

Рисунок 2 — Контейнер PostgreSQL

В записи 5458:5432 слева указан порт компьютера, справа — порт PostgreSQL внутри контейнера. Подключение тома к каталогу данных позволяет хранить файлы базы отдельно от контейнера. Статус Up означает, что контейнер запущен.

Готовность базы проверена утилитой pg_isready. Ответ accepting connections подтверждает, что сервер принимает подключения (рисунок 3).

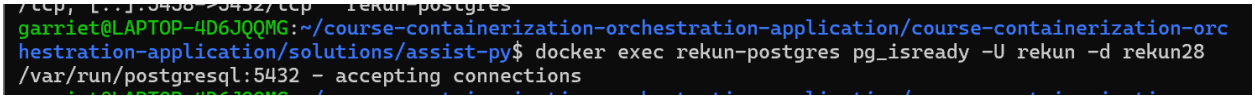


Рисунок 3 — Готовность PostgreSQL к подключениям

Команда docker exec выполняет проверку в работающем контейнере. Утилита pg_isready проверяет, принимает ли сервер подключения; параметры -U и -d задают пользователя и базу. Это более конкретная проверка готовности PostgreSQL, чем наличие статуса Up.

1.2 Подключение FastAPI к PostgreSQL

В приложение добавлены зависимость psycopg2-binary и модуль app/database.py. При запуске создаётся таблица notes, если её ещё нет. Обработчик POST /notes записывает заметку в базу, а GET /notes возвращает сохранённые записи. Параметры соединения передаются через переменные окружения.

После сборки образа rekun-assistant:lab3 приложение запущено с публикацией порта 8028:5458. Выполнены четыре запроса: к главной странице, к маршруту /health, на создание заметки и на получение списка. Новая заметка «ЛР3: данные в БД» получила идентификатор 3. В списке отображаются все три сохранённые записи (рисунок 4).

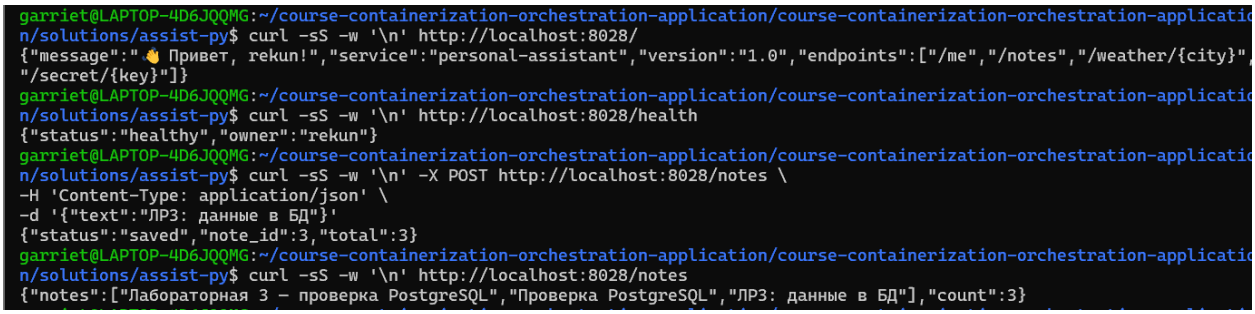


Рисунок 4 — Проверка приложения и сохранения заметки

curl отправляет HTTP-запросы приложению. GET получает данные, POST создаёт запись; тело POST передаётся в формате JSON. Ответ healthy показывает работу маршрута проверки приложения, а успешные запись и чтение заметки подтверждают взаимодействие с базой.

2 ПРОВЕРКА СОХРАНЕНИЯ ДАННЫХ

Для проверки сохранения данных контейнер rekun-assistant-lab3 остановлен и удалён. На рисунке 5 показаны команды удаления при первоначальной проверке. Контейнер PostgreSQL и именованный том продолжили хранить данные.

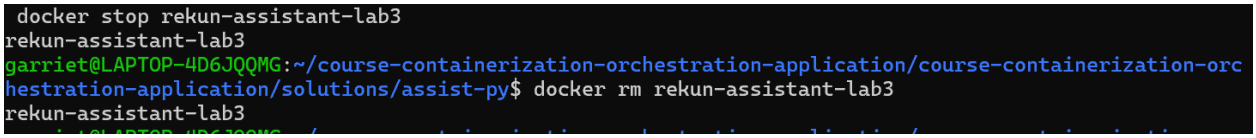


Рисунок 5 — Удаление контейнера приложения

docker stop останавливает процесс контейнера, docker rm удаляет сам контейнер. Образ приложения остаётся и используется для нового запуска. Здесь удаляется только FastAPI: PostgreSQL продолжает работать, поэтому сохранённые в нём записи не теряются.

После добавления трёх заметок контейнер приложения повторно пересоздан. Он подключён к сети rekun-backend с прежними параметрами доступа к PostgreSQL. Запрос GET /notes вернул все три записи. На рисунке 6 видны дата и время проверки, логин, имя компьютера и список заметок.

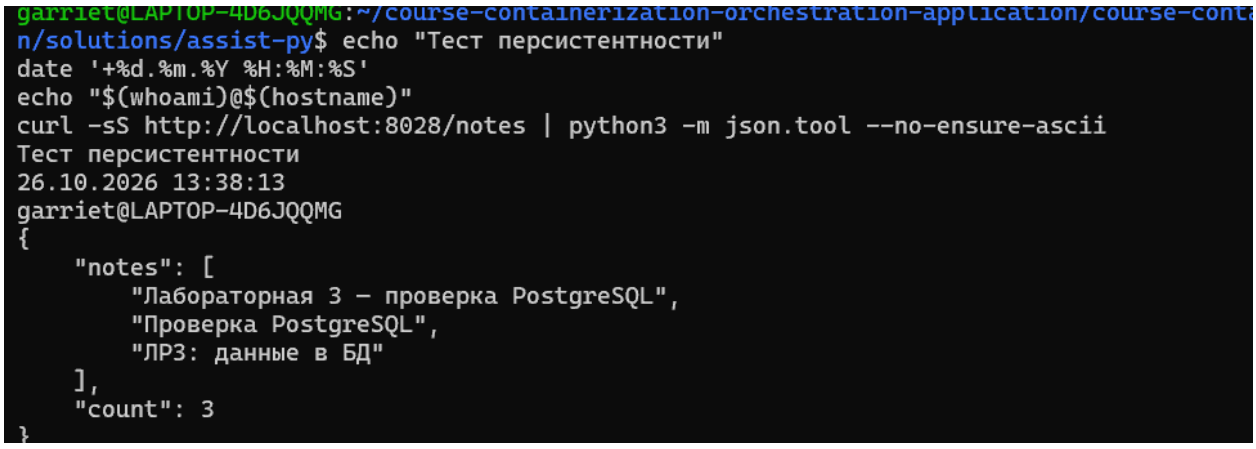


Рисунок 6 — Заметка после пересоздания приложения

Персистентность означает сохранение данных независимо от времени жизни приложения. Если бы заметки оставались только в списке Python, новый процесс начал бы с пустого списка. Получение трёх прежних записей показывает, что приложение читает их из PostgreSQL.

3 ПОДКЛЮЧЕНИЕ КОНФИГУРАЦИИ

В каталоге проекта расположен файл config/app.conf. Он подключён через bind mount к пути /app/config/app.conf в проверочном контейнере rekun-config-check. Использован режим readonly, который ограничивает доступ контейнера к файлу чтением.

Файл приведён к образцу задания: задано ограничение количества заметок, включён параметр погодного API, указаны maintenance_mode=false и владелец owner=rekun. Содержимое прочитано командой docker exec внутри контейнера (рисунок 7).

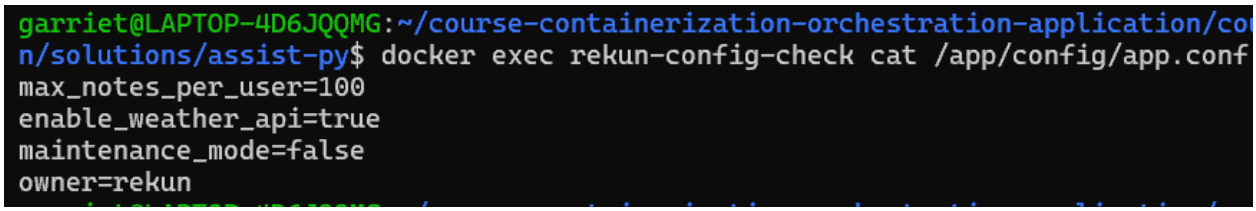


Рисунок 7 — Конфигурация внутри контейнера

При bind mount конкретный файл компьютера становится доступен по заданному пути контейнера. В отличие от именованного тома, здесь явно выбирается исходный путь на хосте. Команда cat выводит содержимое файла; owner указывает владельца учебной конфигурации.

На хосте значение maintenance_mode изменено с false на true. После перезапуска контейнера повторное чтение файла внутри него показало новое значение (рисунок 8). Docker-образ при этом не пересобирался.

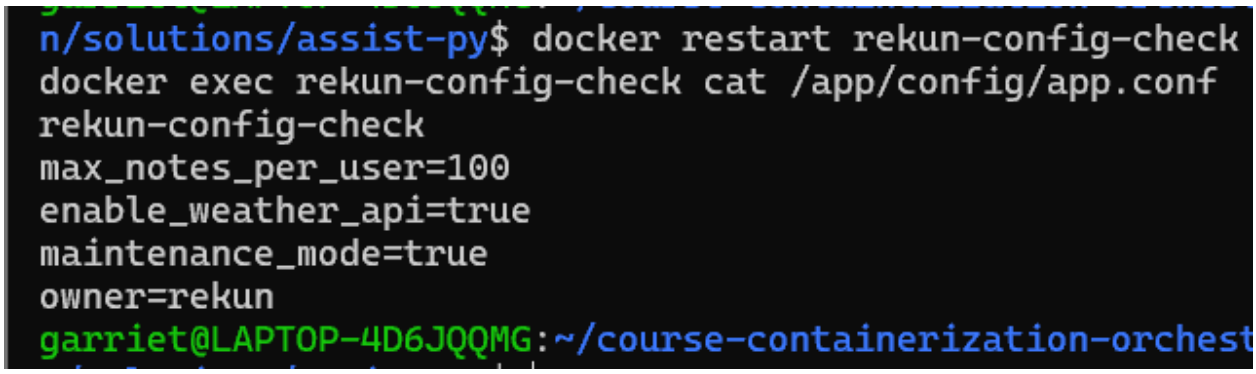


Рисунок 8 — Изменение конфигурации без пересборки

readonly запрещает изменение подключённого файла из контейнера, но позволяет менять его на хосте. docker restart перезапускает существующий контейнер. Файл подключён снаружи, поэтому для изменения его содержимого новая сборка образа не требуется.

В этой реализации файл используется для проверки монтирования; настройки соединения FastAPI с базой передаются через переменные окружения.

4 ПОЛЬЗОВАТЕЛЬСКАЯ СЕТЬ И ИЗОЛЯЦИЯ

4.1 Создание сети и подключение контейнеров

Создана пользовательская сеть rekun-backend с драйвером bridge. Результат создания и наличие сети в списке показаны на рисунке 9.

```
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/
ration-application/solutions/assist-py$ docker network create rekun-backend
59cebbadc31365e24284fedeb208fb2ff2a22952254a315887d980f5c972ac83
garriet@LAPTOP-4D6JQQMG:~/course-containerization-orchestration-application/
ration-application/solutions/assist-py$ docker network ls
NETWORK ID     NAME            DRIVER    SCOPE
08379ccea415   bridge          bridge    local
66b77893ff88   host            host      local
4c3cca764a74   mgmt            bridge    local
78aa55ad7572   mgmt-lab4a      bridge    local
d77d5fd7e843   none            null      local
59cebbadc313   rekun-backend   bridge    local
```

Рисунок 9 — Создание сети rekun-backend

Пользовательская bridge-сеть обеспечивает взаимодействие подключённых контейнеров на одном Docker-хосте. Её имя задаётся при запуске параметром –network. Внутри такой сети Docker позволяет обращаться к другим контейнерам по их именам.

К сети подключены контейнеры базы данных и приложения. Команда docker network inspect выводит имена rekun-postgres и rekun-assistant-lab3 (рисунок 10).

```
ration-application/solutions/assist-py$ docker network inspect rekun-backend --format '{{range .Containers}
}}{{.Name}}{{println}}{{end}}'
rekun-postgres
rekun-assistant-lab3
```

Рисунок 10 — Контейнеры в пользовательской сети

docker network inspect показывает подробные сведения о сети. В данном выводе выбраны только имена подключённых контейнеров. Наличие обоих имён подтверждает их участие в одной сети; реальный обмен данными проверяется отдельно запросом к приложению.

Приложение пересоздано с параметрами DB_HOST=rekun-postgres и DB_PORT=5432. Подключение выполняется по имени контейнера через внутреннюю сеть Docker. При первоначальной проверке запрос к API вернул созданную ранее заметку (рисунок 11).

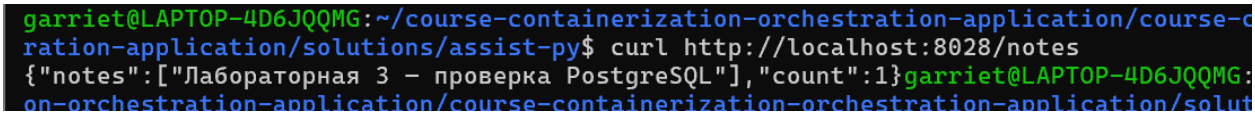


Рисунок 11 — Доступ к заметке после настройки сети

Внутри контейнера localhost обозначает его самого. Поэтому адрес базы задан именем rekun-postgres. Для связи между контейнерами используется внутренний порт PostgreSQL 5432; опубликованный порт хоста 5458 предназначен для подключения через компьютер.

4.2 Проверка связи между контейнерами

Из временного контейнера в сети rekun-backend выполнен ping rekun-postgres. Имя разрешилось в адрес 172.18.0.2; получены ответы на оба пакета, потери составили 0% (рисунок 12).

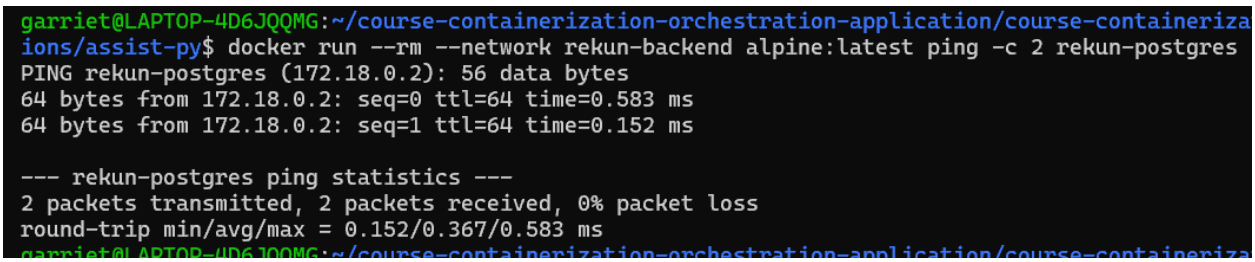


Рисунок 12 — Успешный ping внутри сети

ping проверяет доступность узла пакетами ICMP. Параметр -c 2 ограничивает проверку двумя пакетами. Вывод показывает разрешение имени в IP-адрес и получение двух ответов: 0 процентов потерь означает успешную проверку.

Из временного контейнера в стандартной сети bridge проверен тот же IP-адрес базы. Ответы не получены: потери составили 100% (рисунок 13). Для сопоставления использован фактический адрес PostgreSQL, полученный через docker inspect.

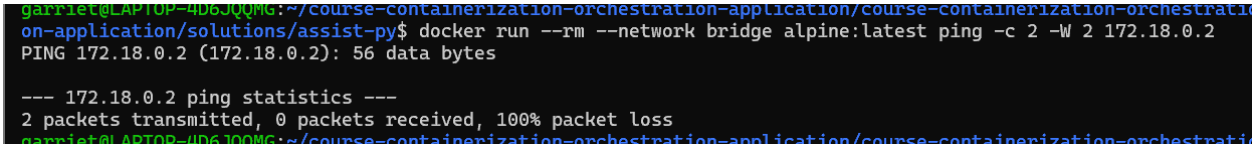


Рисунок 13 — Неудачный ping из сети bridge

Вторая проверка выполняется из другой Docker-сети по фактическому IP базы. Потери 100 процентов означают, что в этом опыте ICMP-ответы не пришли. Этот результат характеризует проверенный маршрут и протокол; он не проверяет доступ через опубликованный порт хоста.

В выполненном опыте ICMP-связь с контейнером базы доступна внутри пользовательской сети и отсутствует из стандартной сети bridge.

5 ИТОГОВАЯ ПРОВЕРКА

Выполнена финальная проверка с указанием даты, времени, логина и имени компьютера. Показаны запущенные контейнеры rekun-postgres, rekun-assistant-lab3 и rekun-config-check, именованный том rekun-pgdata и пользовательская сеть rekun-backend (рисунок 14).

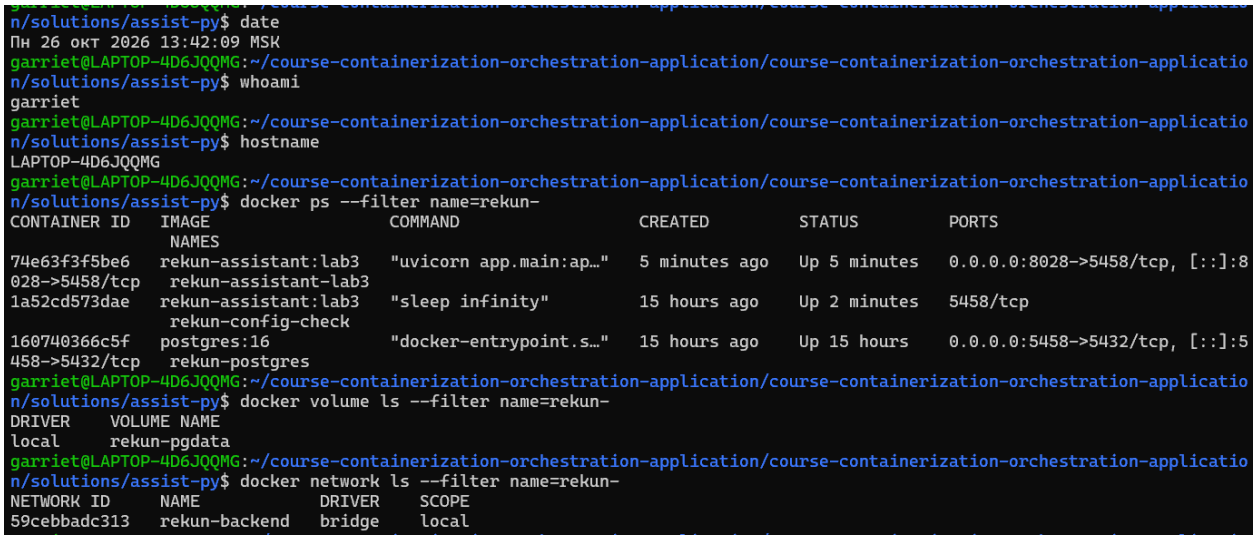


Рисунок 14 — Контейнеры, том и сеть Docker

docker ps показывает работающие контейнеры, docker volume ls — тома, docker network ls — сети. Фильтр name=rekun- оставляет объекты с персональным префиксом. Дата, логин и хост позволяют связать итоговую проверку с окружением студента.

В списке контейнеров видны публикации портов: 8028:5458 для FastAPI и 5458:5432 для PostgreSQL. Наличие тома и сети подтверждено отдельными командами Docker.

В репозитории сохранены исходный код, файл конфигурации и инструкция по воспроизведению работы. В README описаны создание тома и сети, запуск PostgreSQL и FastAPI, проверка заметок, bind mount и сетевого взаимодействия.

ЗАКЛЮЧЕНИЕ

К приложению FastAPI подключена база PostgreSQL, заметки перенесены в таблицу базы данных. Проверено сохранение заметки после удаления и пересоздания контейнера приложения.

Для файлов PostgreSQL создан именованный том. Конфигурационный файл подключён через bind mount в режиме чтения. Проверено обновление файла внутри контейнера после изменения на хосте без пересборки образа. Контейнеры приложения и базы объединены в пользовательскую сеть; проверены связь внутри неё и отсутствие ICMP-ответов из стандартной сети bridge.